Event Processing and Automation Decision Guide

Use this guide to choose where processing runs, what starts it, and whether callers wait for the result. Compare the options and their implementation responsibilities, then follow the linked technical docs.

Companion to Implementing Event Processing, Bots, and Subscriptions. Use the clinical workflow guides to define the business process and what completion means.

Make these choices **for each processing flow**. One application might invoke a Bot to calculate a result, use a Subscription to create follow-up work, and run an existing service on a schedule.

1. **Choose where processing runs.** Use a Medplum Bot or an external service.
2. **Choose the trigger.** Use direct invocation, a Medplum resource change, an event from another system, or a schedule.
3. **Choose whether the caller waits** for directly invoked work and incoming event handlers.

For every flow, define completion and recovery: what proves the work finished, how failures are handled, and how retries avoid duplicate effects. A scheduled run or resource-change handler can skip Decision 3.

Decision 1: Where will your processing run?

Choose who operates the code that performs the work. This choice is independent of the trigger and whether a caller waits for completion.

Option A: Medplum Bots

Start with Bots for processing centered on Medplum resources. A Bot runs your code with a Medplum client, configured permissions, and project secrets.

- **Advantages:**
  - Integrates with Medplum invocation, Subscriptions, scheduling, and execution logging.

    - On hosted Medplum, your team deploys Bot code without operating a separate application server for that processing.
- **Tradeoffs:**
    - Code and dependencies must fit the configured Bot runtime and its limits.
    - Your team still owns the processing logic, tests, access policies, and recovery behavior. Managed execution does not supply the business workflow's completion or retry rules.
- **Best fit:** Resource transformations, workflow steps, and integrations whose execution requirements fit the Bot environment.

Choose the execution identity explicitly. Bots normally use their own permissions; starting a Bot does not inherently make it use the initiating user's permissions.

Implementation: Bot setup and runtime, Execution authority, Bots in Production.

Option B: An external service

Run processing in your application backend or another service your team operates. It can call the Medplum API and receive Medplum Subscription notifications.

- **Advantages:**
    - Reuses an existing backend, worker system, and operational tooling.
    - Lets your team choose its language, dependencies, infrastructure, and execution model.
- **Tradeoffs:**
    - Your team operates the service, credentials, event receiver, and any queue or scheduler.
    - Medplum notification delivery and the service's processing are separate. A successful delivery does not prove that queued work finished.
- **Best fit:** You already operate the relevant backend, or need execution capabilities outside the Bot environment.

Choose whether the service acts with its own permissions or with an individual user's access. Service credentials alone do not give requests that user's permissions or attribution.

Implementation: External services, Execution authority, Client Credentials.

Next for either option: Choose what starts this processing flow.

Decision 2: How will processing be triggered?

Choose the event that represents an intention to do work. Avoid triggering the same business action through several paths unless they share a stable operation identity.

If a rule must reject an action, enforce it in the request path before the action is committed. The post-save Subscriptions described here cannot reject the original write.

Ensure every permitted write path enforces the rule; callers must not be able to bypass a controlled endpoint with unrestricted direct writes.

Option A: Direct invocation

Your application explicitly asks a Bot or service to perform an action. For Bots, use $execute, or expose the Bot through a custom FHIR operation when a named operation is useful to your API.

- **Advantages:**
  - The action and its inputs are explicit, and the caller can receive a result or validation failure.
  - Fits a controlled path for actions that require business checks before writing data.
- **Tradeoffs:**
  - Every caller must use the agreed path. A Bot or custom operation does not automatically intercept ordinary FHIR writes.
  - Repeated requests still need a policy for identifying the same action.
- **Best fit:** User-requested actions, calculations, and commands issued by another application.

Implementation: Direct invocation, Bot $execute, Custom FHIR Operations.

Next: Continue to Decision 3.

Option B: A Medplum resource change

Use a scoped Subscription to invoke a Bot or notify an external service after a matching resource changes.

- **Advantages:**
  - Processing can respond to changes from multiple writers, including applications and integrations.
  - The original write does not wait for downstream processing to finish.
- **Tradeoffs:**
  - Downstream processing can fail after the write succeeds. Represent and monitor that outcome separately where it matters to the workflow.
  - Handlers must account for repeated delivery and changes that occur before processing runs. Subscriptions are not a permanent, ordered event archive.
- **Best fit:** Follow-up work, projections, and notifications that should react to committed data.

Should every matching update run, or only a specific transition?

- **Every matching update:** Use when the output must be refreshed as relevant data changes. Filter out irrelevant changes.

- **A specific transition:** Use when work should start once a resource enters a state, such as a preparation Task becoming completed. A filter for status=completed alone also matches later edits while the Task remains completed. Use interaction and FHIRPath criteria to distinguish the transition. Decide separately whether creation in that state should trigger work.

If processing writes resources, scope the work type, interaction, and transition so those writes do not trigger the same work again or create a loop through another Subscription. Decide how to handle an event whose source has since changed, such as work that was cancelled before the handler ran.

Implementation: Resource-change processing example, Subscription Extensions.

Next: Skip Decision 3 and define completion and recovery.

Option C: An event from another system

Receive an event from an external system, commonly through its webhook. The receiver can be a Bot or your service. This is distinct from a Medplum Subscription delivering a notification to your service.

- **Advantages:**
  - Responds to a source event without waiting for a polling schedule.
- **Tradeoffs:**
  - The source defines delivery, acknowledgment, and retry behavior. Verify its authentication and payload contract.
  - Missed events may require a separate reconciliation process; receiving a webhook does not establish ownership or conflict rules between systems.
- **Best fit:** A source system offers events that identify the work your application needs to perform.

Implementation: Receiving external events, Consuming Webhooks. Use the relevant integration docs for the source's data mapping and lifecycle.

Next: Continue to Decision 3, using the source's response deadline and acknowledgment rules.

Option D: A schedule

Run a Bot or service periodically to find eligible work, such as overdue items, due occurrences, or records requiring reconciliation.

- **Advantages:**
  - Supports time-based work and sources that do not provide events.
  - Can catch up work that was missed by another trigger when the necessary state is retained.
- **Tradeoffs:**
  - Freshness depends on the schedule and processing capacity.

    - Define missed-run catch-up, overlapping execution, time zones, and stop conditions. A schedule alone does not prevent duplicate work.
- **Best fit:** Periodic reconciliation, recurring work, and deadline-based processing.

Implementation: Scheduled processing, Cron Jobs for Bots.

Next: Skip Decision 3 and define completion and recovery.

Decision 3: Should the caller wait for processing to finish?

For direct invocation and incoming event handlers. Choose what the response promises. Resource-change processing already happens after the original write; a schedule has no interactive caller waiting for its result.

Option A: Complete processing before responding

The caller waits for a result or failure.

- **Advantages:**
    - The caller can use the result immediately or respond to a validation error.
- **Tradeoffs:**
    - Processing must fit the caller's and execution environment's time limits.
    - A timeout may leave an uncertain outcome. Waiting does not make multiple writes or external actions atomic, and a later failure does not automatically roll back earlier work.
- **Best fit:** The caller needs the result to continue and the work can finish within the request's time budget.

Implementation: Direct invocation. For multiple writes, choose a supported write pattern. Atomic transaction Bundles are currently unavailable on hosted Medplum. If partial completion would violate a business rule, revisit the flow before implementation.

Option B: Accept the work and process it in the background

The response confirms acceptance. The caller or an operator determines completion separately.

- **Advantages:**
    - The caller can continue while processing runs.
    - Fits work that does not need an immediate result or takes longer than the caller can wait.
- **Tradeoffs:**
    - Acceptance and completion need distinct meanings, with retained state and a recovery path appropriate to the reliability requirement.
    - An asynchronous response does not itself guarantee automatic retry or recovery after interruption.
- **Best fit:** Delayed completion is acceptable and your application can expose and recover unfinished work.

For authenticated Bot $execute, Medplum supports asynchronous execution with a status URL. That mechanism provides execution tracking. Recovery after interrupted execution requires retained work state and application logic. Custom FHIR operations and public webhook routes do not support Prefer: respond-async. Use the implementation docs to choose the supported path and determine whether a separate persisted work item is required.

Implementation: Background execution and recovery, Asynchronous Bot execution.

Define completion and recovery

Answer these questions for each flow. The example below uses a Subscription-triggered Bot to create a review Task when a preparation Task becomes completed.

What result proves this flow completed?

In this example, the review Task's existence proves the automation completed. Saving the preparation Task starts the work; completing the human review is a separate step.

Use existing resource or execution state as evidence. Add a work record only when needed to track steps or retain inputs.

Implementation: Completion state for background work.

What happens if processing fails or its outcome is unknown?

If the Bot fails, the preparation Task remains completed. Define how to find preparations missing their reviews, who is responsible for recovery, and whether a worker or an operator runs it. Monitor unfinished work as well as execution errors.

If an external action times out, determine whether it succeeded before resending, or use the provider's supported idempotency mechanism. Retain the inputs and correlation identifiers needed to investigate and recover.

Implementation: Recovery, Monitoring.

How can the work be retried without duplicating its effects?

Give the intended action a stable identity. In this example, repeated processing should reuse the review Task for the same preparation, preserving its assignee and progress. Use conditional creation for generated resources and version checks when competing attempts update existing resources.

Protect external actions separately: reusing a FHIR resource does not prevent a duplicate email or submission.

Implementation: Repeatable processing, Review Task example and recovery code.

Record the result, action identity, and recovery owner. Test failure and retry paths before enabling production triggers.